The Backlog System

Complete Deployment Tutorial

How the system works, and how to set it up on a new Claude Code project from scratch. Written for someone deploying it for the first time.

Created by Christian Taylor · linkedin.com/in/mrchristiantaylor

1. What this system is

The problem it solves. When you're building with Claude Code, new ideas and “feature creep” constantly surface mid-build — a fix here, a feature there. Interrupting Claude Code to add each one derails the work it's focused on; but if you don't capture them, they're forgotten. This system gives those stray ideas a home so they're never lost and never interrupt the flow.

The idea in one sentence. You jot ideas into a simple list from anywhere (even your phone), and at natural stopping points Claude Code turns them into well-formed tasks, asks you what to tackle now versus later, does the work, and keeps a running record — all without you ever opening a spreadsheet.

The pieces

- **The backlog (BACKLOG.xlsx).** A spreadsheet that holds your tasks, one per row, with defined columns (what it is, how urgent, its status, notes, and more). You never edit it by hand — Claude Code does.
- **The iCloud inbox (a text file).** A plain text file in iCloud Drive where you drop one-line ideas from your phone or Mac. It's your “capture anywhere” mailbox.
- **The rules (CLAUDE.md).** Instructions that teach Claude Code the whole workflow — how to read the inbox, fill in every column, and run the end-of-sprint routine. This is what makes the automation happen.
- **The dashboard (generate_dashboard.py → dashboard.html).** A clean, read-only web view of the backlog so you can see everything at a glance without opening Excel. It refreshes itself. Sort by date and filter by source — yours vs. Claude’s — with separate counts for each.

How it flows

1. **Capture:** an idea hits — you add a one-liner to the iCloud inbox.
2. **Drain:** at the end of a work session, you tell Claude Code the sprint is done. It reads the inbox and turns each idea into a fully-detailed backlog row.
3. **Decide:** it lists the new (and any pending) items and asks you, for each, whether to do it now or schedule it for later.
4. **Do:** for the “now” items, it does the work in priority order, following the task's built-in instructions.
5. **Record:** it marks each finished item Done, stamps the date, writes a note on what changed, and refreshes the dashboard.

Two Claudes, two roles

Claude Code runs on your Mac and can read and write your project's files — it does the work and updates the backlog. The **Claude Project** (on the web) cannot touch files on your Mac, so it's optional here — useful only as a place to think through a fuzzy idea before you jot it. The iCloud inbox exists precisely because the web can't reach your local files, but a synced text file can.

> **Why a spreadsheet you never open**
>
> The backlog is a spreadsheet because it gives every task tidy, consistent fields. But you interact with it only

through the inbox (to add) and the dashboard (to view). Claude Code is the only one that edits the file — that's what keeps it clean and consistent.

2. Before you start (prerequisites)

- **A Mac with Claude Code installed and working** in the project you want to use this with.
- **Python 3.** It powers the dashboard generator. Check by running `python3 --version` in Terminal — if you see a version number, you're set. If not, install Apple's tools with `xcode-select --install`.
- **A project in a clean location.** Not inside Documents, Desktop, or Downloads (macOS locks those down) and not iCloud-synced. A folder like `~/dev/your-project` is ideal. Avoid spaces in folder names — use hyphens.
- **Two reusable files from this toolkit:** BACKLOG.xlsx (the template, which includes a built-in test item) and generate_dashboard.py. Keep master copies somewhere safe so you can drop them into any new project.
- **Full Disk Access for your terminal.** The app you run Claude Code from (Terminal, iTerm, or Cursor's built-in terminal) must have Full Disk Access turned on in macOS System Settings — without it, Claude Code can't read the iCloud inbox. Step 6 covers how.

3. Deploy it, step by step

Do these in order. Commands go in the Terminal app (open it from Spotlight: Command + Space, type Terminal). Run one at a time. Replace your-project with your actual project folder name.

Step 1 Put your project in a clean location

If your project is new or currently lives in Documents/Desktop/Downloads, place it under ~/dev with a hyphenated name (no spaces):

```
mkdir -p ~/dev
```

For a brand-new project, create it there. For an existing one, see the separate "Moving Your Project" guide — always copy, never move, so the original stays safe.

Step 2 Create the BACKLOG folder

```
mkdir -p ~/dev/your-project/BACKLOG
```

Step 3 Add the two files

Copy BACKLOG.xlsx and generate_dashboard.py into that BACKLOG folder (drag them in via Finder, or use the command below adjusted to where your master copies live):

```
cp ~/Downloads/BACKLOG.xlsx ~/Downloads/generate_dashboard.py ~/dev/your-project/BACKLOG/
```

Step 4 Make sure Python can run it

Move into the project and test the generator:

```
cd ~/dev/your-project
python3 BACKLOG/generate_dashboard.py
```

If it says openpyxl is missing, install it once and run the command again:

```
pip3 install openpyxl --break-system-packages
```

Step 5 Create the iCloud inbox (one time)

```
touch "$HOME/Library/Mobile Documents/com~apple~CloudDocs/backlog-inbox.txt"
```

This makes an empty text file in your iCloud Drive. To add ideas from your phone: Files app → iCloud Drive → tap backlog-inbox.txt → type one idea per line.

Step 6 Turn on Full Disk Access for your terminal

Your iCloud inbox lives in a folder macOS protects, so the app you run Claude Code from needs Full Disk Access, or it can't read the inbox.

Open System Settings → Privacy & Security → Full Disk Access. Turn on the app you use for Claude Code — Terminal, iTerm, or `Cursor` if you run Claude Code in its built-in terminal. Then fully quit and reopen that app — the setting only takes effect after a restart.

> **Why this is needed**
> The project itself lives in ~/dev (not protected), but the inbox file lives in iCloud Drive, which macOS locks down. Full Disk Access is what lets Claude Code reach across to read and clear the inbox.

Step 7 Add the rules to CLAUDE.md

Open (or create) a file named `CLAUDE.md` at the root of your project, and paste in the complete block from **Appendix A** at the end of this tutorial. That block is what teaches Claude Code the entire workflow.

Step 8 Generate and open the dashboard

```
python3 BACKLOG/generate_dashboard.py
open BACKLOG/dashboard.html
```

You should see one item (the built-in test, BL-000) under “Not Started.” Bookmark this page and leave the tab open — it refreshes itself every 45 seconds.

Step 9 One-time setup with Claude Code

Start Claude Code in the project and say: **“Read the backlog rules in CLAUDE.md and do the one-time setup.”** It will confirm it has stored the workflow in its memory (and update WAYS_OF_WORKING.md if you keep one).

Step 10 Run the smoke test (a safe dry run)

Tell Claude Code: **“We just finished a sprint — run the end-of-sprint backlog check.”** Confirm it behaves like this:

1. Lists the BL-000 test item and does NOT start it — it asks whether to do it now or later.
2. You answer **“Do it now.”**
3. It runs the test (which only creates a throwaway file, touching no code), marks it Done, stamps the date, writes a note, and refreshes the dashboard.
4. Your dashboard moves BL-000 to “Done.” Confirm no real code changed.

> **This is your proof it works**
> The BL-000 test exercises the entire loop safely — no application code is touched. Once it completes and your dashboard updates, the system is working end to end.

Step 11 Go live

Delete the BL-000 row if you like (or keep it as a record), then start capturing real ideas into the inbox. You're running.

4. Using it day to day

Capturing ideas

Whenever something comes up, add a one-liner to the iCloud inbox — from your phone (Files app) or your Mac. One idea per line. Add as much or as little detail as you want; Claude Code fills in the rest.

Draining the inbox

At the end of a work session, tell Claude Code: **"We just finished a sprint — run the end-of-sprint backlog check."** It processes the inbox into backlog rows, clears the inbox, then walks you through what to do now versus later.

Viewing the backlog

Keep the dashboard tab open. It refreshes on its own after Claude Code makes changes; you never open the spreadsheet.

Adding something on the spot

When you're at your Mac and want an item added immediately, just say **"add to the backlog: …"** to Claude Code — no inbox needed.

5. The columns, explained

Column	What it means	Who fills it
ID	Unique tag like BL-014, for reference	Claude Code
Task	One-line summary of the work	Claude Code
Type	Feature / Bug / Refactor / Chore	Claude Code
Priority	High / Medium / Low	Claude Code
Status	Not Started / In Progress / Blocked / Done	Claude Code
Source	Me (you asked) / Claude (Claude added it)	Claude Code
Sprint	Target sprint, or Unscheduled	Claude Code
Component	App area (auth, API, UI, …)	Claude Code
Details	Acceptance criteria, specifics, scope	Claude Code
Depends On	Prerequisite item IDs	Claude Code
Date Added	When it entered the backlog	Claude Code
Date Completed	When it was finished	Claude Code (on done)
Kick-Off Prompt	The exact instruction to build it	Claude Code
Claude Notes	What was done, files, follow-ups	Claude Code (on done)

You never fill any of these yourself — you supply a one-line idea and Claude Code produces the whole row.

6. Choosing the Type

- **Feature —** new functionality that doesn't exist yet.
- **Bug —** something already built that behaves wrong.
- **Refactor —** improving working code without changing what the user sees.
- **Chore —** upkeep: dependencies, config, tooling, docs.

Quick test: Does it work right now? If not → Bug. If it works but you want it cleaner → Refactor. Brand-new user-facing thing → Feature. Behind-the-scenes upkeep → Chore. (Claude Code picks this for you; this is here so you can sanity-check.)

7. Troubleshooting

If you see…	It means…	Do this
zsh: command not found: python	Your Mac uses python3, not python	Use python3 in the command
Operation not permitted	Project is in a locked/synced folder (Documents, iCloud)	Move it to ~/dev (see Moving guide)
openpyxl is required	The one helper library isn't installed	pip3 install openpyxl --break-system-packages
Claude Code says the sheet won't open	BACKLOG.xlsx is open in Excel (locked)	Close it in Excel, then retry
Dashboard shows old data	It hasn't refreshed / wasn't regenerated	Wait 45s or click Refresh now; make sure Claude Code ran the generator
Claude Code says the inbox is empty (but it isn't)	The terminal running Claude Code lacks Full Disk Access, so it can't read the iCloud inbox	Turn on Full Disk Access for that app (System Settings › Privacy & Security), then quit and reopen it
An inbox idea wasn't added	It was too vague to turn into a task	Claude Code left it and will ask you to clarify

Appendix A — The complete CLAUDE.md block

Paste everything below into the CLAUDE.md file at your project root (Step 6). This is the full set of rules that runs the system.

```
## Backlog workflow

The backlog lives at BACKLOG/BACKLOG.xlsx, inside this project directory. The top-level
directory is intentionally NOT a git repo — only certain subdirectories are version-controlled.

Row columns, left to right: ID, Task, Type, Priority, Status, Source, Sprint, Component,
Details, Depends On, Date Added, Date Completed, Kick-Off Prompt, Claude Notes.
- Kick-Off Prompt is your instruction to execute (source of truth for the work).
- Claude Notes and Date Completed are yours to fill when you finish an item.
- Status values: Not Started, In Progress, Blocked, Done.
- Source values: "Me" (I requested it) or "Claude" (you added it on your own initiative).
- Sprint holds a target sprint (e.g., "Sprint 7") or "Unscheduled".
- Depends On lists prerequisite IDs (e.g., "BL-002") that must be Done before this can start.

I view the backlog ONLY through a read-only dashboard at BACKLOG/dashboard.html, generated by
BACKLOG/generate_dashboard.py. I do not open the .xlsx. So the dashboard must be regenerated
after any change you make to the sheet.

### One-time setup (first time you read these rules)
1. Persist this workflow to your project memory so it survives across sessions — keep these
   sections in CLAUDE.md and note that the backlog is driven by BACKLOG/BACKLOG.xlsx with the
   dashboard generated by BACKLOG/generate_dashboard.py.
2. If we maintain a WAYS_OF_WORKING.md in this project, append a short section describing this
   process for humans. If it doesn't exist, skip this. Tell me once done; don't repeat setup.

### Hard rules
- NEVER run git init in the top-level project directory or in BACKLOG/.
- NEVER add BACKLOG.xlsx or dashboard.html to any repo's tracking, and never move them into a
  version-controlled subdirectory. The spreadsheet stays untracked. If you think version
  control is needed, ask me first.
- generate_dashboard.py only READS the xlsx and WRITES dashboard.html. It never edits the sheet.
```

```
## Backlog inbox — process at the START of every sprint-end check

I capture ideas as one-line entries in an iCloud text file so I can add them from my phone:

    ~/Library/Mobile Documents/com~apple~CloudDocs/backlog-inbox.txt

It lives in iCloud Drive, OUTSIDE the project. Read and write it at that path. Never move it
into the project or add it to git. (Reading it requires the terminal to have Full Disk Access.)

Before running the end-of-sprint check, process the inbox:
1. Read backlog-inbox.txt. If empty or missing, skip to the sprint-end check.
2. For each non-empty line that is a real, actionable idea, author a complete row using the
   Authoring rules below and append it to BACKLOG/BACKLOG.xlsx with Status = "Not Started",
   Source = "Me", Sprint = "Unscheduled", Date Added = today, ID = next free BL-###.
3. Remove a line ONLY after its row is confirmed written. If BACKLOG.xlsx can't be written
   (e.g., open in Excel), stop, leave ALL lines in place, and tell me — never delete an
   unsaved idea.
4. If a line is too vague to become a real task, leave it and ask me. Don't guess or delete.
5. Report what you added and the new IDs (e.g., "Added 3: BL-014, BL-015, BL-016"), then
   regenerate the dashboard: python3 BACKLOG/generate_dashboard.py
6. Added items are now "Not Started" rows, so they appear in the sprint-end review below.

You may also add an item on demand anytime I say "add to the backlog: <idea>" (Source = "Me").

### Authoring rules (fill EVERY column correctly)
- ID: next free BL-### (never reuse).
- Task: one-line imperative summary ("Add…", "Fix…").
- Type: Feature (new functionality) / Bug (built but behaves wrong) / Refactor (improve working
  code, no user-facing change) / Chore (deps, config, tooling, docs). If it both improves AND
  changes behavior, it's Feature or Bug, not Refactor.
- Priority: High (blocks work/users) / Medium (important, not urgent) / Low (nice-to-have).
  Default to Medium if I gave no urgency signal.
- Status: "Not Started".
- Source: "Me" for items I requested (inbox, brainstorm, or "add to the backlog"); "Claude" for
  items you add on your own initiative (see below).
- Sprint: "Unscheduled" unless my note names a sprint.
- Component: app area it touches (auth, API, UI, billing). Infer if obvious; ask only if it
  genuinely matters and isn't clear.
- Details: acceptance criteria, edge cases, file paths, patterns to match, out-of-scope notes.
- Depends On: prerequisite IDs if I named any; otherwise empty.
- Date Added: today (YYYY-MM-DD).  Date Completed: empty.
- Kick-Off Prompt: a complete, self-contained instruction you can later execute — goal + scope,
  acceptance criteria, files to touch and to avoid, conventions to match, ending with: "Verify
  the change, set this row's Status to Done, stamp Date Completed, and record what you did in
  the Claude Notes column."
- Claude Notes: empty (you fill this when done).

### Adding your own items
If, while working, you identify a genuinely necessary follow-up task (a discovered bug, a
required refactor, a dependency), you may add it to the backlog as a new row with Source =
"Claude", Status = "Not Started", authored by the rules above — then tell me you added it. Only
add warranted items; don't pad the backlog. These show up separately on my dashboard so I can
see what you flagged versus what I asked for.

## End-of-sprint backlog check
1. Read BACKLOG/BACKLOG.xlsx. If it won't open, tell me it's likely open in Excel and stop.
2. Find every row with Status = "Not Started". If none, tell me the backlog is clear.
3. List them compactly (ID, Task, Type, Priority, Source, Sprint). Do NOT start any yet.
4. Ask me for each: do it now, or schedule it for a future sprint?
5. Before starting a "do now" item, check Depends On. If any prerequisite isn't Done, set the
   item to "Blocked", note what it waits on, and don't start it.
6. For "do now" items (dependencies satisfied): execute in Priority order, following the
   Kick-Off Prompt exactly. When each is verified, set Status = Done, stamp Date Completed, and
   write a Claude Notes entry (what changed, files touched, follow-ups, commit/branch).
7. For "later" items: set Sprint to the target I name, leave Status "Not Started".
8. Whenever you changed the sheet, regenerate the dashboard: python3 BACKLOG/generate_dashboard.py
   (if openpyxl is missing: pip3 install openpyxl --break-system-packages).

Never begin a backlog item without my go-ahead on timing.
```